

FACULDADE ESAMC UBERLÂNDIA  
PROJETO DE EXTENSÃO - 1 SEMESTRE DE 2026

COORDENADOR DE EXTENSÃO: Prof. Yassushi Okada

DISCIPLINA: Redes de Computadores II

CURSOS: Sistemas de Informação

EQUIPE:

|                                       |
|---------------------------------------|
| Andressa da Silva Santos              |
| Gabriel Pedroso Naves Monteiro        |
| Matheus Alves Bonetti                 |
| Pedro Valêncio Souza Marques Moutinho |
| João Pedro Ferreira Alves             |

Andressa da Silva Santos  
Gabriel Pedroso Naves Monteiro  
Pedro Valêncio Souza Marques Moutinho  
Matheus Alves Bonetti  
João Pedro Ferreira Alves

#### DOCUMENTOR

Inteligência Artificial aplicada ao auxílio educacional e preparação para o ENADE

Relatório apresentado ao  
curso de graduação  
em Sistemas de Informação  
da ESAMC,  
como requisito parcial do  
Projeto de Extensão  
da disciplina Redes de  
Computadores II.

Orientador(a): Prof(a).  
Yassushi Okada

**Uberlândia - MG**  
**2026**

## **RESUMO**

O presente projeto apresenta o desenvolvimento do sistema “DocuMentor”, uma Inteligência Artificial educacional criada com o objetivo de auxiliar estudantes em seus processos de aprendizagem, especialmente na preparação para o Exame Nacional de Desempenho dos Estudantes (ENADE).

O sistema foi desenvolvido para oferecer suporte acadêmico através de funcionalidades como resolução de dúvidas, explicações de conteúdos, utilização de questões anteriores do ENADE, auxílio em estudos personalizados e geração de perguntas relacionadas aos temas estudados pelos usuários.

Além de auxiliar na preparação para avaliações acadêmicas, o DocuMentor também busca democratizar o acesso ao aprendizado, oferecendo uma ferramenta tecnológica acessível para estudantes que possuem dificuldades de estudo, interpretação de conteúdos e organização acadêmica.

O desenvolvimento do projeto possibilitou a aplicação prática de conhecimentos relacionados à programação, inteligência artificial, desenvolvimento de software e experiência do usuário, integrando tecnologia e impacto social em uma única solução.

## SUMÁRIO

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>SUMÁRIO.....</b>                                      | <b>4</b>  |
| <b>1. INTRODUÇÃO.....</b>                                | <b>5</b>  |
| <b>1.1 INTELIGÊNCIA ARTIFICIAL NA EDUCAÇÃO.....</b>      | <b>6</b>  |
| <b>1.2 O ENADE E AS DIFICULDADES DOS ESTUDANTES.....</b> | <b>6</b>  |
| <b>2. OBJETIVOS.....</b>                                 | <b>7</b>  |
| 2.1 OBJETIVO GERAL.....                                  | 7         |
| 2.2 OBJETIVOS ESPECÍFICOS.....                           | 7         |
| <b>3. REFERENCIAL TEÓRICO.....</b>                       | <b>8</b>  |
| 3.1 INTELIGÊNCIA ARTIFICIAL APLICADA AO ENSINO.....      | 8         |
| 3.2 TECNOLOGIAS EDUCACIONAIS.....                        | 8         |
| 3.3 APRENDIZAGEM ASSISTIDA POR IA.....                   | 8         |
| <b>4. METODOLOGIA.....</b>                               | <b>9</b>  |
| 4.1 PLANEJAMENTO DO PROJETO.....                         | 9         |
| 4.2 DESENVOLVIMENTO DO SISTEMA.....                      | 9         |
| Figura 1 — Estrutura inicial do projeto.....             | 9         |
| 4.3 ESTRUTURA DA APLICAÇÃO.....                          | 10        |
| Figura 2 — Organização do código-fonte.....              | 10        |
| <b>5. DESENVOLVIMENTO DO PROJETO.....</b>                | <b>14</b> |
| 5.1 ESTRUTURA DO SISTEMA.....                            | 14        |
| Figura 3 — Interface inicial do sistema.....             | 14        |
| 5.2 FUNCIONALIDADES DA IA.....                           | 14        |
| Figura 4 — Sistema de perguntas e respostas.....         | 15        |
| 5.3 INTERFACE DO USUÁRIO.....                            | 17        |
| Figura 5 — Desenvolvimento da interface.....             | 17        |
| 5.4 INTEGRAÇÃO DAS FUNCIONALIDADES.....                  | 18        |
| Figura 6 — Integração do sistema.....                    | 18        |
| <b>6. RESULTADOS E DISCUSSÕES.....</b>                   | <b>19</b> |
| <b>7. CONCLUSÃO.....</b>                                 | <b>20</b> |
| <b>8. REFERÊNCIAS BIBLIOGRÁFICAS.....</b>                | <b>21</b> |

## 1. INTRODUÇÃO

A tecnologia vem transformando significativamente a forma como estudantes aprendem e acessam informações. Entre essas transformações, destaca-se o avanço da Inteligência Artificial (IA), que passou a ser utilizada em diferentes áreas da sociedade, incluindo a educação.

Ferramentas inteligentes vêm sendo utilizadas para auxiliar estudantes no aprendizado, proporcionando explicações rápidas, conteúdos personalizados e apoio em atividades acadêmicas. Dentro desse contexto, surgiu o projeto “DocuMentor”, desenvolvido com o objetivo de criar uma plataforma de auxílio educacional baseada em Inteligência Artificial.

O sistema possui como principal foco auxiliar estudantes na preparação para o ENADE, utilizando questões anteriores, explicações detalhadas, auxílio em dúvidas e suporte ao aprendizado. Entretanto, a plataforma também pode ser utilizada em outras áreas de estudo, ampliando sua aplicação acadêmica.

Além do desenvolvimento tecnológico, o projeto também possui caráter social, buscando auxiliar estudantes que possuem dificuldades de aprendizagem, acesso limitado a materiais de estudo ou necessidade de reforço acadêmico.

## **1.1 INTELIGÊNCIA ARTIFICIAL NA EDUCAÇÃO**

A Inteligência Artificial aplicada à educação vem crescendo nos últimos anos devido à sua capacidade de personalizar o aprendizado e auxiliar estudantes em tempo real.

Sistemas inteligentes conseguem analisar perguntas, fornecer respostas contextualizadas e adaptar conteúdos conforme a necessidade do usuário. Dessa forma, a IA se torna uma ferramenta importante no apoio educacional, principalmente em ambientes acadêmicos e universitários.

O uso dessas tecnologias também contribui para tornar o estudo mais acessível, dinâmico e interativo, aproximando os estudantes da tecnologia de maneira positiva.

---

## **1.2 O ENADE E AS DIFICULDADES DOS ESTUDANTES**

O Exame Nacional de Desempenho dos Estudantes (ENADE) é uma avaliação aplicada aos estudantes do ensino superior com o objetivo de medir o desempenho acadêmico dos cursos universitários no Brasil.

Muitos estudantes enfrentam dificuldades durante a preparação para o exame, principalmente relacionadas à interpretação de conteúdos, organização dos estudos e acesso a materiais de apoio.

Diante desse cenário, o DocuMentor foi pensado como uma ferramenta capaz de auxiliar nesse processo, oferecendo questões, explicações e suporte educacional através de Inteligência Artificial.

## **2. OBJETIVOS**

### **2.1 OBJETIVO GERAL**

Desenvolver uma plataforma baseada em Inteligência Artificial capaz de auxiliar estudantes em seus estudos, especialmente na preparação para o ENADE, oferecendo suporte educacional através de perguntas, explicações e conteúdos acadêmicos.

---

### **2.2 OBJETIVOS ESPECÍFICOS**

- Desenvolver uma interface intuitiva e acessível;
- Criar um sistema capaz de responder dúvidas acadêmicas;
- Auxiliar estudantes na preparação para o ENADE;
- Disponibilizar questões e explicações educativas;
- Incentivar o uso da tecnologia na educação;
- Promover inclusão e apoio acadêmico para estudantes com dificuldades de aprendizagem.

### **3. REFERENCIAL TEÓRICO**

#### **3.1 INTELIGÊNCIA ARTIFICIAL APLICADA AO ENSINO**

A Inteligência Artificial consiste em sistemas computacionais capazes de simular comportamentos humanos, como interpretação, aprendizado e resolução de problemas.

Na educação, essas tecnologias podem ser utilizadas para criar ambientes de aprendizagem personalizados, auxiliando estudantes na resolução de dúvidas e no desenvolvimento acadêmico.

---

#### **3.2 TECNOLOGIAS EDUCACIONAIS**

As tecnologias educacionais possuem papel importante na modernização do ensino, permitindo maior acessibilidade e dinamismo nos estudos.

Plataformas digitais, sistemas inteligentes e ferramentas online ampliam o acesso à informação e contribuem para uma aprendizagem mais eficiente.

---

#### **3.3 APRENDIZAGEM ASSISTIDA POR IA**

Sistemas educacionais baseados em IA conseguem adaptar conteúdos conforme as necessidades do usuário, oferecendo explicações mais rápidas e suporte contínuo ao estudante.

Esse modelo de aprendizado vem sendo utilizado como apoio complementar ao ensino tradicional.



## 4. METODOLOGIA

### 4.1 PLANEJAMENTO DO PROJETO

O desenvolvimento do projeto iniciou-se através de pesquisas relacionadas à Inteligência Artificial, plataformas educacionais e dificuldades enfrentadas por estudantes universitários durante a preparação para avaliações acadêmicas.

Após os estudos iniciais, foram definidos os objetivos do sistema, suas funcionalidades e a estrutura geral da aplicação.

---

### 4.2 DESENVOLVIMENTO DO SISTEMA

O sistema “DocuMentor” foi desenvolvido utilizando a linguagem de programação Python no back-end, devido à sua ampla utilização em aplicações de Inteligência Artificial, processamento de dados e desenvolvimento de sistemas inteligentes.

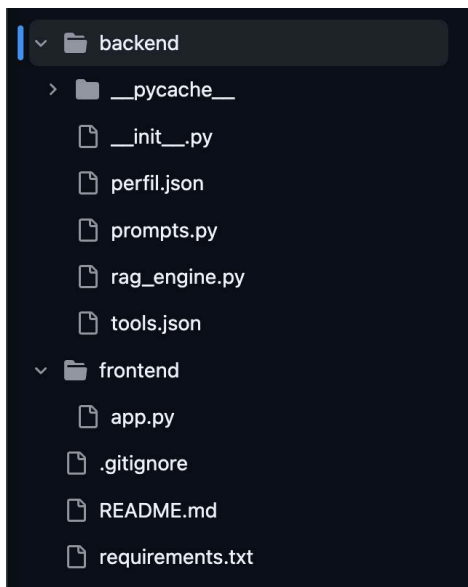
Para o desenvolvimento da interface visual da plataforma foi utilizado o framework Streamlit, ferramenta voltada para criação de aplicações web interativas em Python de forma simples e eficiente.

A utilização do Streamlit permitiu criar uma interface intuitiva e funcional, possibilitando que os usuários interajam facilmente com a Inteligência Artificial através da plataforma.

Durante o desenvolvimento do sistema foram realizadas etapas de:

- planejamento;
- modelagem;
- programação;
- testes;
- ajustes da interface;
- implementação das funcionalidades principais.

#### **Figura 1 — Estrutura inicial do projeto**

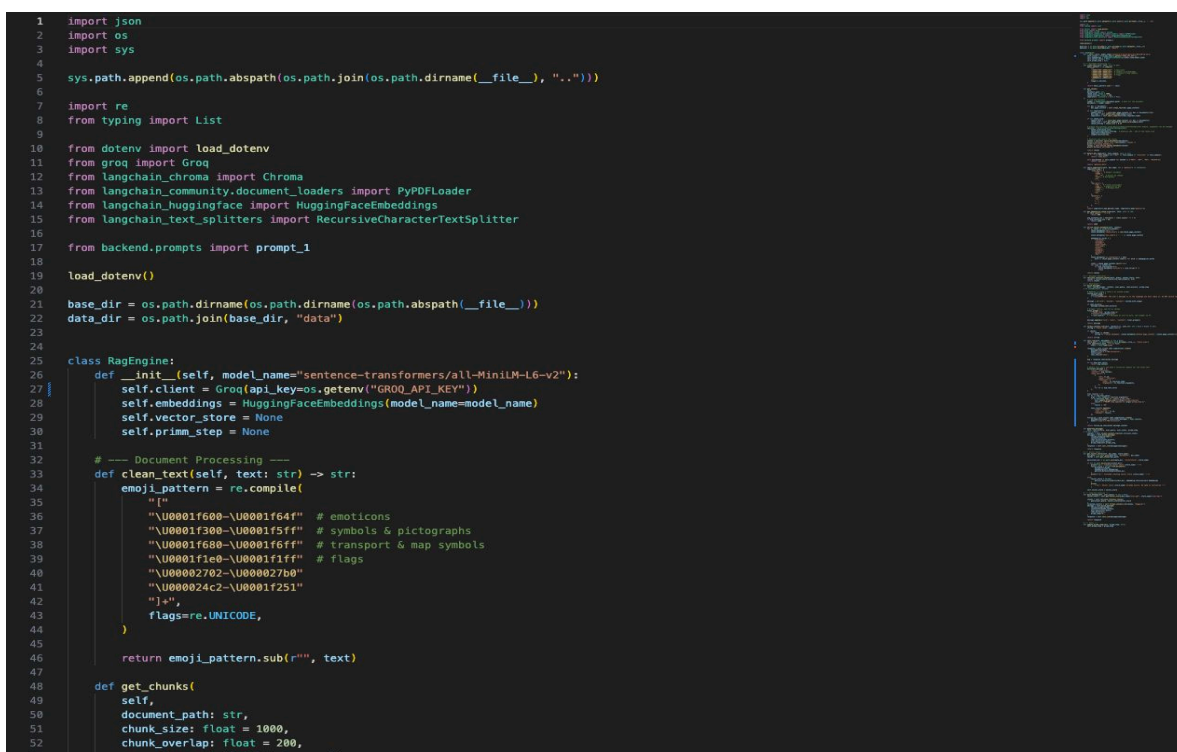


### 4.3 ESTRUTURA DA APLICAÇÃO

A aplicação foi organizada de maneira modular, permitindo melhor separação entre as funcionalidades do sistema e facilitando futuras manutenções e melhorias.

O back-end desenvolvido em Python ficou responsável pela lógica principal do sistema, processamento das perguntas realizadas pelos usuários e integração com os recursos de Inteligência Artificial.

Figura 2 — Organização do código-fonte



```

53     separators: List[str] | None = None,
54 ):
55     # Load the document
56     loader = PyPDFLoader(document_path) # path for the document
57     documents = loader.load()
58
59     for doc in documents:
60         doc.page_content = self.clean_text(doc.page_content)
61
62     if not separators:
63         sample_text = " ".join([doc.page_content for doc in documents[:3]])
64         doc_type = self.detect_doc_type(sample_text)
65         separators = self.smart_separators(doc_type=doc_type)
66
67     if not chunk_size:
68         sample_text = " ".join([doc.page_content for doc in documents])
69         chunk_size = self.get_adaptative_chunk_size(sample_text)
70         chunk_overlap = chunk_size * 0.13
71
72     # Create the splitter using RecursiveCharacterTextSplitter module, arguments can be changed
73     splitter = RecursiveCharacterTextSplitter(
74         chunk_size=chunk_size,
75         chunk_overlap=chunk_overlap, # Generally 10% - 20% of the chunk size
76         separators=separators,
77         length_function=len,
78     )
79
80     # Generate and return the chunks
81     chunks = splitter.split_documents(documents)
82     print(f"Documents split into {len(chunks)} chunks.")
83     print("Enriching metadata.")
84     chunks = self.enrich_chunks_metadata(chunks)
85     print("Metadata enriched.")
86
87     return chunks
88
89 def detect_doc_type(self, text_sample: str) -> str:
90     if "```" in text_sample and ("def" in text_sample or "function" in text_sample):
91         return "code_docs"
92
93     elif any(method in text_sample for method in ["POST", "GET", "PUT", "DELETE"]):
94         return "api_docs"
95
96     return "general_docs"
97
98 def smart_separators(self, doc_type: str = "general") -> List[str]:
99     separators_map = {
100         "code_docs": [

```

```

100         "code_docs": [
101             "\n## ", # Headers markdown
102             "\n### ",
103             "\n```\n", # Blocos de código
104             "\n\n", # Parágrafos
105             "\n",
106             ". ",
107         ],
108         "api_docs": [
109             "\n#", # Títulos principais
110             "\n#", # Endpoints/seções
111             "\nPOST ", # Métodos HTTP
112             "\nGET ",
113             "\n\n",
114             "\n",
115         ],
116         "general": [
117             "\n\n",
118             "\n",
119             ". ",
120             "! ",
121             "? ",
122         ],
123     }
124     return separators_map.get(doc_type, separators_map["general"])
125
126 def get_adaptative_chunk_size(self, text: str) -> int:
127     if text.count("```") > 5:
128         return 800
129
130     avg_sentence_len = len(text) / (text.count(".") + 1)
131     if avg_sentence_len > 50:
132         return 1200
133
134     return 1000
135
136 def enrich_chunks_metadata(self, chunks):
137     for i, chunk in enumerate(chunks):
138         chunk.metadata["id"] = i
139         chunk.metadata["chunk_size"] = len(chunk.page_content)
140
141         chunk.metadata["has_code"] = "```" in chunk.page_content
142
143         pedagogical_words = [
144             "conceito",
145             "concept",
146             "entender",
147             "understand",

```

```

148         "funciona",
149         "works",
150         "exemplo",
151         "example",
152         "porque",
153         "why",
154     ]
155     chunk.metadata["is_conceptual"] = any(
156         word in chunk.page_content.lower() for word in pedagogical_words
157     )
158
159     lines = chunk.page_content.split("\n")
160     for line in lines[:3]:
161         if line.startswith("#"):
162             chunk.metadata["section"] = line.strip("# ")
163             break
164
165     return chunks
166
167 # --- Document Retrieval ---
168 def retrieve_relevant_chunks(self, query, vector_store, k=5):
169     chunks = vector_store.similarity_search(query, k=k)
170     return chunks
171
172 # --- Generation ---
173 def build_message(
174     self, system_prompt, context, user_query, chat_history, primm_step
175 ) -> list[dict[str, None]]:
176
177     # Detecta a língua e reforça no system prompt
178     system_with_lang = (
179         system_prompt
180         + f"\n\nIMPORTANT: The user's message is in the language you must reply in. Do NOT switch languages under any circumstance."
181     )
182
183     message = [{"role": "system", "content": system_with_lang}]
184
185     if chat_history:
186         message.extend(chat_history)
187
188     # Wrapper neutro, sem forçar idioma
189     final_prompt = (
190         f"PRIMM Step: {primm_step}\n"
191         f"Context:\n(context)\n\n"
192         f"{user_query}" # - Mensagem do usuário pura, sem wrapper em PT
193     )
194

```

```

194         message.append({"role": "user", "content": final_prompt})
195
196     return message
197
198 def format_context_llm(self, chunks=None, user_lvl: str | None = None) -> str:
199     string = f"User level: {user_lvl}\n"
200
201     if chunks:
202         for chunk in chunks:
203             string += f"Chunk metadata: {chunk.metadata}\nChunk page_content: {chunk.page_content}\n"
204
205     return string
206
207 def call_llm(self, messages) -> str | None:
208     tools_path = os.path.join(os.path.dirname(__file__), "tools.json")
209     with open(tools_path, "r") as file:
210         tools = json.load(file)
211
212     response = self.client.chat.completions.create(
213         messages=messages,
214         model="llama-3.3-70b-versatile",
215         tools=tools,
216         tool_choice="auto",
217     )
218
219     msg = response.choices[0].message
220
221     if not msg.tool_calls:
222         return msg.content
223
224     # Handle tool calls and make a follow-up request for the final text
225     assistant_message = {
226         "role": "assistant",
227         "content": msg.content,
228         "tool_calls": [
229             {
230                 "id": tc.id,
231                 "type": "function",
232                 "function": {
233                     "name": tc.function.name,
234                     "arguments": tc.function.arguments,
235                 },
236             },
237             for tc in msg.tool_calls
238         ],
239     }
240
241     tool_results = []
242     for tc in msg.tool_calls:
243

```

```

244     args = json.loads(tc.function.arguments)
245     if tc.function.name == "update_primm_step":
246         self.update_primm_step(str(args["primm_step"]))
247         result = f"PRIMM step updated to {args['primm_step']}"
248     else:
249         result = "OK"
250
251     tool_results.append({
252         "role": "tool",
253         "tool_call_id": tc.id,
254         "content": result,
255     })
256
257     follow_up = self.client.chat.completions.create(
258         messages=messages + [assistant_message] + tool_results,
259         model="llama-3.3-70b-versatile",
260     )
261
262     return follow_up.choices[0].message.content
263
264 def generate_message(
265     self, chat_history, user_query, user_level, primm_step
266 ) -> str | None:
267     context = self.format_context_llm(user_lvl=user_level)
268     messages = self.build_message(
269         system_prompt=prompt_1,
270         context=context,
271         chat_history=chat_history,
272         user_query=user_query,
273         primm_step=self.primm_step,
274     )
275     response = self.call_llm(messages=messages)
276
277     return response
278
279 # --- Vector Store ---
280 def get_vector_store(self, doc_name, store_name):
281     doc_path = os.path.join(data_dir, "documents", doc_name)
282     chunks = self.get_chunks(doc_path)
283
284     persistent_dir = os.path.join(data_dir, "vectorstore", store_name)
285
286     if not os.path.exists(persistent_dir):
287         print(f"\n--- Creating vector store {store_name} ---")
288         vector_store = Chroma.from_documents(
289             documents=chunks,
290             embedding=self.embeddings,
291             persist_directory=persistent_dir,
292         )

```

```

293         print(f"\n--- Finished creating vector store {store_name} ---")
294
295     else:
296         vector_store = Chroma(
297             persist_directory=persistent_dir, embedding_function=self.embeddings
298         )
299         print(
300             f"\n--- Vector store {store_name} already exists. No need to initialize ---"
301         )
302
303     self.vector_store = vector_store
304
305 # --- Complete Pipeline ---
306 def main_method(self, user_query) -> str | None:
307     vector_store = self.get_vector_store(doc_name="eloz.pdf", store_name="eloz-api")
308
309     chunks = self.retrieve_relevant_chunks(
310         query=user_query, vector_store=vector_store
311     )
312     formatted_context = self.format_context_llm(chunks, "beginner")
313     message = self.build_message(
314         system_prompt=prompt_1,
315         context=formatted_context,
316         user_query=user_query,
317         chat_history=[],
318         primm_step="0",
319     )
320     response = self.call_llm(messages=message)
321
322     return response
323
324 # --- Tools ---
325 def update_primm_step(self, primm_step: str):
326     self.primm_step = primm_step
327

```

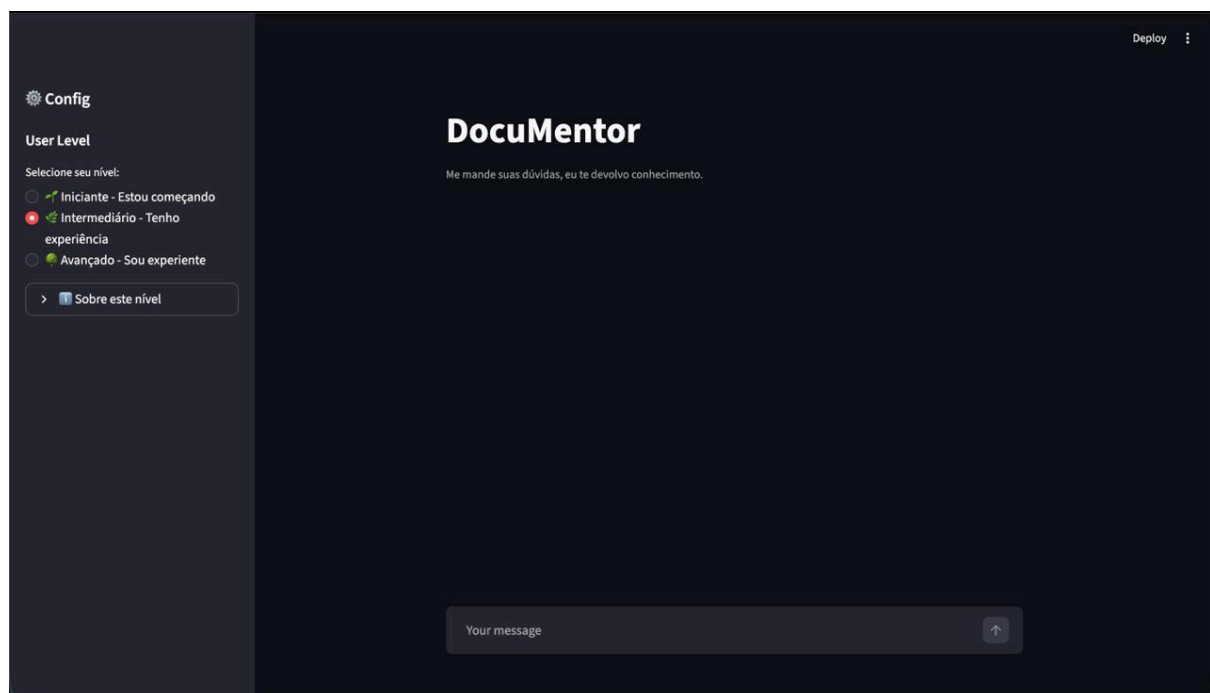
## 5. DESENVOLVIMENTO DO PROJETO

### 5.1 ESTRUTURA DO SISTEMA

O DocuMentor foi desenvolvido com foco em simplicidade e acessibilidade, permitindo que estudantes consigam utilizar a plataforma de maneira intuitiva.

O sistema conta com uma interface de interação entre usuário e Inteligência Artificial, possibilitando o envio de perguntas e recebimento de respostas relacionadas aos conteúdos acadêmicos.

**Figura 3 — Interface inicial do sistema**



---

### 5.2 FUNCIONALIDADES DA IA

Entre as principais funcionalidades implementadas no sistema estão:

- auxílio em dúvidas acadêmicas;
- explicações de conteúdos;
- suporte aos estudos para o ENADE;
- perguntas educativas;
- utilização de questões anteriores;
- interação com Inteligência Artificial.



## Figura 4 — Sistema de perguntas e respostas



Me ajude a estudar para o Enade, quero as questões mais frequentes e com os assuntos mais frequentes também, corrija cada uma me dizendo exatamente aonde errei e passando mais exercícios para que eu possa melhorar a minha lógica



Olá João Pedro! Vamos começar a estudar para o ENADE.

Eu sei que você está procurando pelas questões mais frequentes e com os assuntos mais frequentes também. Aqui estão 3 questões antigas do ENADE que você pode tentar:

**Questão 1** Um sistema de gerenciamento de estoque utiliza uma lista para armazenar os produtos em estoque. Cada produto é representado por um dicionário com as chaves "nome", "quantidade" e "preco". Escreva uma função que receba como entrada uma lista de produtos e retorne a quantidade total de produtos em estoque.

Exemplo de entrada:

```
produtos = [
    {"nome": "Produto A", "quantidade": 10, "preco": 10.0},
    {"nome": "Produto B", "quantidade": 20, "preco": 20.0},
    {"nome": "Produto C", "quantidade": 30, "preco": 30.0}
]
```

Qual é a saída da função para a entrada acima?

Aguarde enquanto eu aguardo sua resposta...

(Lembre-se de que estamos no Step 4 - MODIFY do método PRIMM, então você deve aplicar seus conhecimentos para resolver as questões)

Você pode me passar sua resposta agora.



Estou estudando para o Enade e estou com dúvidas no assunto de Estrutura de dados, gostaria que me ajudasse a entender como funciona o conceito de pilhas. Quero que me passe exercícios.



Olá João Pedro! Estou aqui para ajudá-lo a entender o conceito de pilhas em Estrutura de Dados.

Uma pilha é uma estrutura de dados que segue o princípio LIFO (Last-In-First-Out), ou seja, o último elemento que é adicionado à pilha é o primeiro a ser removido.

Aqui está um exemplo simples de como uma pilha funciona:

```
pilha = []
pilha.append(1) # adiciona 1 à pilha
pilha.append(2) # adiciona 2 à pilha
pilha.append(3) # adiciona 3 à pilha
print(pilha) # imprime [1, 2, 3]
print(pilha.pop()) # imprime 3 e remove 3 da pilha
print(pilha) # imprime [1, 2]
```

Agora, vamos fazer alguns exercícios para você praticar:

**Exercício 1** Criar uma função que adiciona elementos à pilha e imprime a pilha após cada adição.

```
def adiciona_elementos(pilha, elementos):
    # sua implementação aqui
    pass

pilha = []
elementos = [1, 2, 3, 4, 5]
adiciona_elementos(pilha, elementos)
print(pilha)
```



Irei fazer o Enade e não sei o que irá cair na prova, pode me passar um resumo e alguns exercícios para que eu saiba em que nível eu estou e como eu posso melhorar?



Olá João Pedro! Estou aqui para ajudá-lo a se preparar para o ENADE.

#### Resumo do ENADE

O ENADE é uma prova que avalia a capacidade dos estudantes de aplicar conceitos e habilidades em diferentes áreas da Ciência da Computação. A prova é dividida em duas partes: a primeira parte é uma prova objetiva, com 30 questões de múltipla escolha, e a segunda parte é uma prova discursiva, com 2 questões que devem ser respondidas em forma de texto.

#### Áreas de Conhecimento

A prova do ENADE cobre as seguintes áreas de conhecimento:

1. **Estruturas de Dados:** listas, pilhas, filas, árvores, grafos
2. **Algoritmos:** ordenação, busca, manipulação de strings
3. **Programação:** linguagens de programação, controle de fluxo, funções
4. **Banco de Dados:** modelagem, consultas, normalização
5. **Sistemas Operacionais:** conceitos básicos, processos, threads
6. **Redes de Computadores:** conceitos básicos, protocolos, segurança

#### Exercícios

Aqui estão alguns exercícios para você testar seus conhecimentos:

**Exercício 1** Implemente uma função que ordene uma lista de números inteiros em ordem crescente.

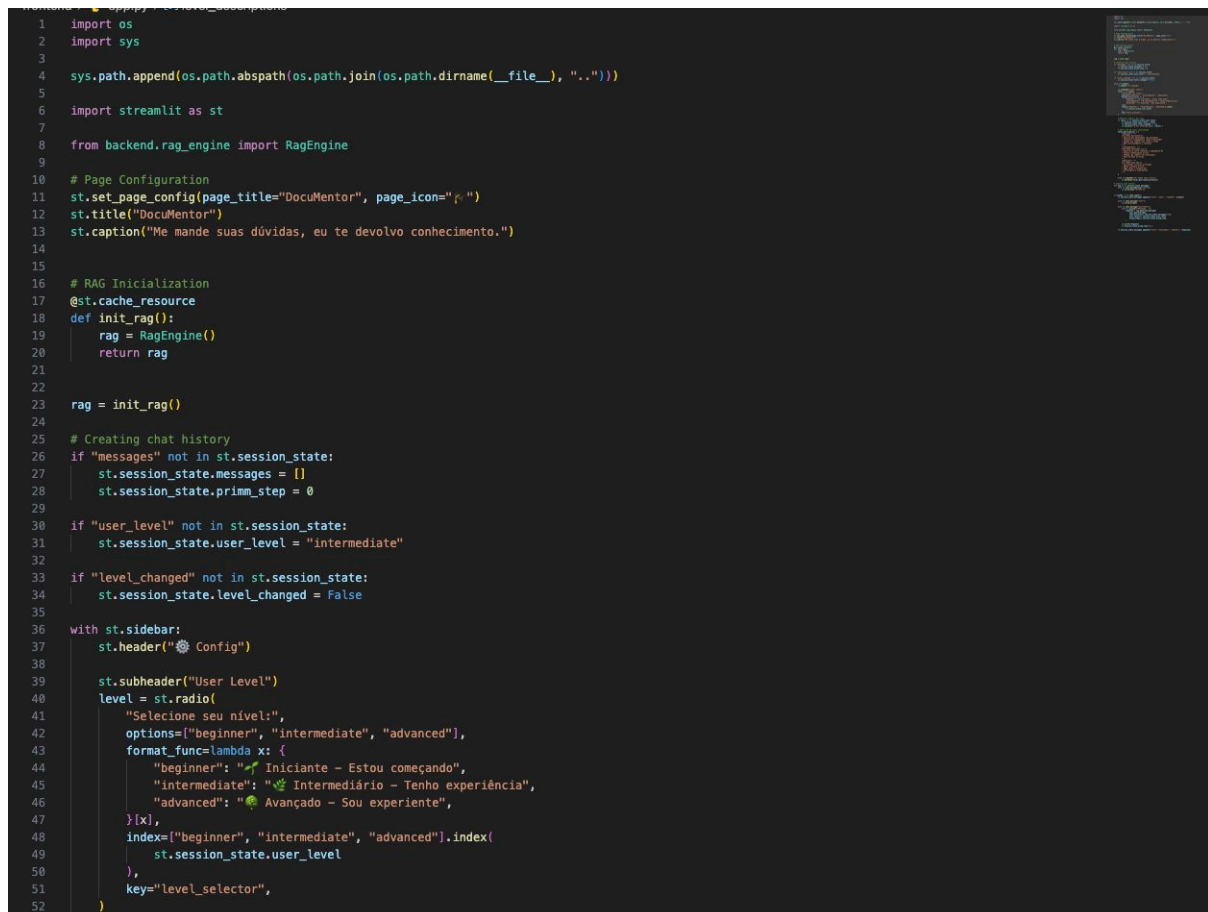


## 5.3 INTERFACE DO USUÁRIO

A interface do sistema foi construída utilizando Streamlit, framework que possibilita o desenvolvimento rápido de aplicações web interativas utilizando Python.

O objetivo foi criar um ambiente simples, organizado e intuitivo, permitindo que estudantes consigam utilizar a plataforma de maneira prática durante seus estudos.

**Figura 5 — Desenvolvimento da interface**



```
1 import os
2 import sys
3
4 sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))
5
6 import streamlit as st
7
8 from backend.rag_engine import RagEngine
9
10 # Page Configuration
11 st.set_page_config(page_title="DocuMentor", page_icon="📖")
12 st.title("DocuMentor")
13 st.caption("Me mande suas dúvidas, eu te devolvo conhecimento.")
14
15 # RAG Initialization
16 @st.cache_resource
17 def init_rag():
18     rag = RagEngine()
19     return rag
20
21 rag = init_rag()
22
23 # Creating chat history
24 if "messages" not in st.session_state:
25     st.session_state.messages = []
26     st.session_state.prim_step = 0
27
28 if "user_level" not in st.session_state:
29     st.session_state.user_level = "intermediate"
30
31 if "level_changed" not in st.session_state:
32     st.session_state.level_changed = False
33
34 with st.sidebar:
35     st.header("⚙️ Config")
36
37     st.subheader("User Level")
38     level = st.radio(
39         "Selecione seu nível:",
40         options=["beginner", "intermediate", "advanced"],
41         format_func=lambda x: {
42             "beginner": "👶 Iniciante - Estou começando",
43             "intermediate": "🧑 Intermediário - Tenho experiência",
44             "advanced": "🦹 Avançado - Sou experiente",
45         }[x],
46         index=["beginner", "intermediate", "advanced"].index(
47             st.session_state.user_level
48         ),
49     ),
50     key="level_selector",
51 )
```

```

53
54 # Detectar mudança de nível
55 if level != st.session_state.user_level:
56     st.session_state.user_level = level
57     st.session_state.level_changed = True
58     st.success(f"Nível alterado para: {level}")
59
60 # Descrição do nível selecionado
61 level_descriptions = {
62     "beginner": """
63     **🌱 Modo Iniciante:**
64     - Explicações detalhadas com analogias
65     - Conceitos fundamentais sempre explicados
66     - Código com comentários linha a linha
67     - Mais encorajamento e contexto
68     """,
69     "intermediate": """
70     **🌱 Modo Intermediário:**
71     - Equilíbrio entre conceito e implementação
72     - Assume conhecimento básico
73     - Código com comentários principais
74     - Foco em boas práticas
75     """,
76     "advanced": """
77     **🌱 Modo Avançado:**
78     - Discussões técnicas profundas
79     - Menos contexto básico
80     - Edge cases e otimizações
81     - Performance e arquitetura
82     """,
83 }
84
85 with st.expander("📖 Sobre este nível"):
86     st.markdown(level_descriptions[level])
87
88 # Showing chat history
89 for msg in st.session_state.messages:
90     with st.chat_message(msg["role"]):
91         st.write(msg["content"])
92
93
94 if prompt := st.chat_input():
95     st.session_state.messages.append({"role": "user", "content": prompt})
96
97     with st.chat_message("user"):
98         st.write(prompt)
99
100     with st.chat_message("assistant"):
101         with st.spinner("Pensando..."):
102             response = rag.generate_message(
103                 user_query=prompt,
104                 chat_history=st.session_state.messages[:-1],
105
106                 chat_history=st.session_state.messages[:-1],
107                 user_level=st.session_state.user_level,
108                 primm_step=st.session_state.primm_step,
109             )
110
111             st.write(response)
112             st.session_state.primm_step += 1
113
114     st.session_state.messages.append({"role": "assistant", "content": response})

```

## 5.4 INTEGRAÇÃO DAS FUNCIONALIDADES

Após o desenvolvimento individual das funcionalidades, foram realizados testes para verificar o funcionamento correto do sistema e a comunicação entre os componentes da aplicação.

**Figura 6 — Integração do sistema**

```

○ joaoapedro@MacBook-Air-de-Joao DocuMentor % streamlit run frontend/app.py
2026-05-21 19:42:02.145 Uvicorn server started on 0.0.0.0:8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://10.23.2.73:8501

```

## **6. RESULTADOS E DISCUSSÕES**

O desenvolvimento do projeto permitiu aplicar conhecimentos relacionados à programação, desenvolvimento de software e Inteligência Artificial em uma solução com foco educacional.

O sistema apresentou resultados positivos durante os testes realizados, demonstrando capacidade de auxiliar estudantes através de respostas rápidas, explicações e apoio aos estudos.

Além do aspecto tecnológico, o projeto também apresentou relevância social, considerando que muitos estudantes possuem dificuldades de aprendizagem e acesso limitado a ferramentas educacionais de qualidade.

O DocuMentor demonstra como a tecnologia pode ser utilizada como ferramenta de inclusão acadêmica e apoio ao desenvolvimento estudantil.

## 7. CONCLUSÃO

O desenvolvimento do projeto “DocuMentor” possibilitou a criação de uma ferramenta tecnológica voltada ao auxílio educacional, utilizando Inteligência Artificial como suporte aos estudantes durante seus processos de aprendizagem.

A plataforma foi desenvolvida utilizando Python no back-end e Streamlit no front-end, permitindo a construção de uma aplicação funcional, acessível e intuitiva. A integração dessas tecnologias possibilitou desenvolver um sistema capaz de responder dúvidas, auxiliar nos estudos e oferecer suporte acadêmico de maneira prática e interativa.

O principal foco do projeto foi auxiliar estudantes na preparação para o ENADE, utilizando perguntas, explicações e apoio educacional através da Inteligência Artificial. Entretanto, a plataforma também pode ser aplicada em diferentes áreas do aprendizado acadêmico.

Além da experiência técnica adquirida durante o desenvolvimento do sistema, o projeto também proporcionou uma reflexão sobre o impacto social da tecnologia na educação, demonstrando como ferramentas inteligentes podem auxiliar estudantes com dificuldades de aprendizagem e ampliar o acesso ao conhecimento.

Dessa forma, conclui-se que o projeto atingiu seus objetivos, integrando tecnologia, educação e impacto social em uma única solução, além de demonstrar na prática a aplicação de Python e Streamlit no desenvolvimento de sistemas educacionais baseados em Inteligência Artificial.

## 8. REFERÊNCIAS BIBLIOGRÁFICAS

[1] Russell, Stuart; Norvig, Peter. Inteligência Artificial. Rio de Janeiro: Elsevier.

[2] Ministério da Educação – ENADE. Disponível em:

<https://www.gov.br/inep>

[3] Tecnologias aplicadas à educação. Disponível em:

<https://www.scielo.br>

[4] GitHub – Plataforma de desenvolvimento colaborativo. Disponível em:

<https://github.com>

[5] Repositório oficial do projeto DocuMentor. Disponível em:

<https://github.com/Vlencio/DocuMentor>